LINMA2710 - Scientific Computing Shared-Memory Multiprocessing

P.-A. Absil and B. Legat

☐ Full Width Mode ☐ Present Mode

Table of Contents

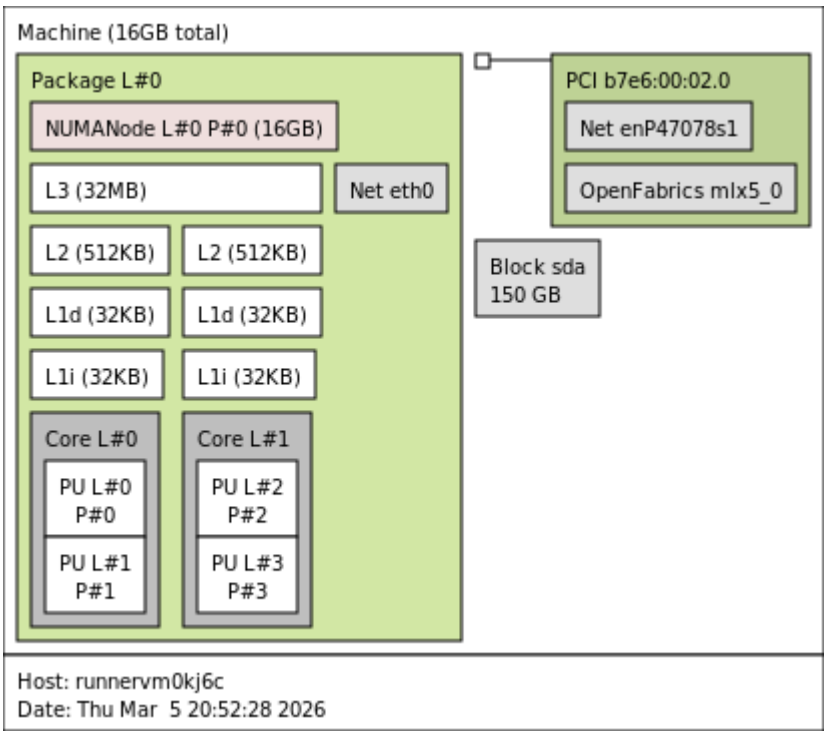
Memory layout

Parallel sum

Amdahl's law

[Eij10] V. Eijkhout. Introduction to High Performance Scientific Computing, 3 Edition, Vol. 1 (Lulu.com, 2010).

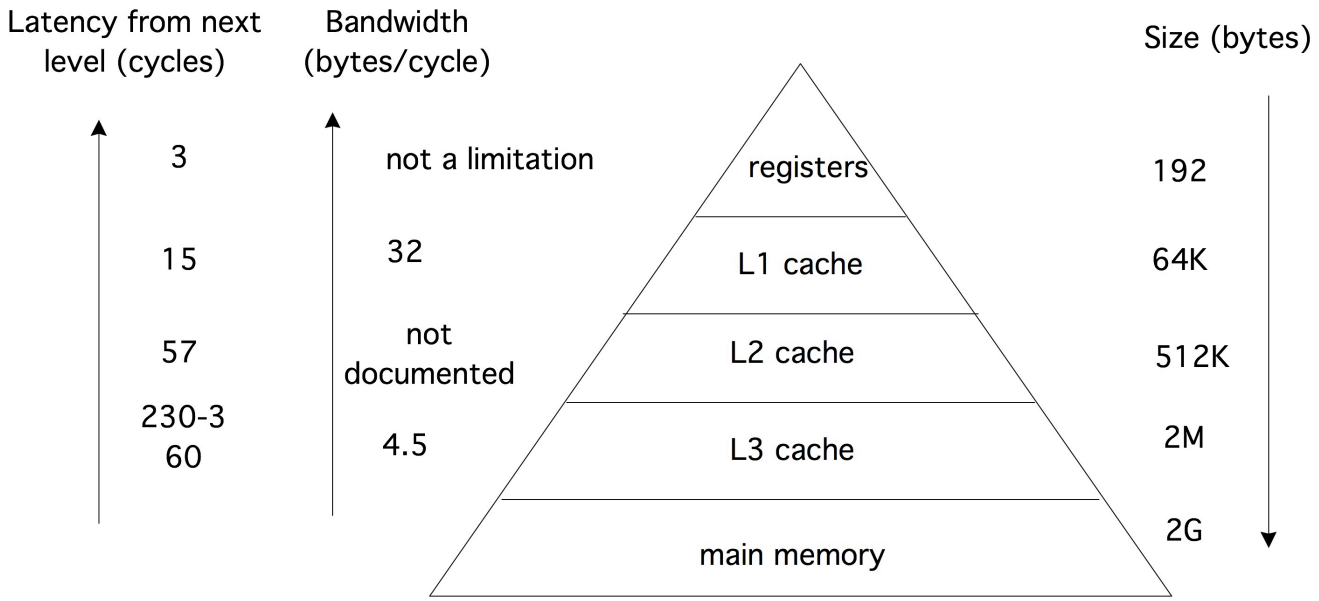
Memory layout



Try it on your laptop!

```
$ lstopo
```

Hierarchy



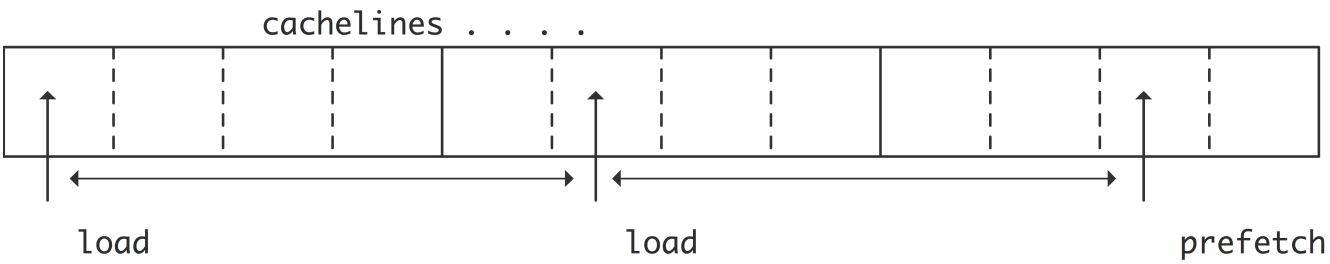
Latency of n bytes of data is given by

$$\alpha + \beta n$$

where α is the start up time and β is the inverse of the bandwidth.

[Eij10; Figure 1.5]

Cache lines and prefetch



- Accessing value not in the cache → *cache miss*
- This value is then loaded along with a whole cache line (e.g., 64 or 128 contiguous bytes)

- Following cache lines may also be anticipated and prefetched

This shows the importance of *data locality*. An algorithm performs better if it accesses data close in memory and in a predictable pattern.

[Eij10; Figure 1.11]

Illustration with matrices

```
32711.197f0
```

```julia
@btime c_sum($mat)
```

```
79.408 μs (0 allocations: 0 bytes)
```

```
mat = 256×256 Matrix{Float32}:
 0.761938  0.779979   0.44236     0.130955   …  0.927359   0.338667   0.291779
 0.59578   0.328133   0.559129    0.151534      0.488626   0.889192   0.704077
 0.665273  0.386817   0.450767    0.511545      0.146696   0.461515   0.750843
 0.229037  0.177665   0.863425    0.640104      0.0584006  0.390661   0.469078
 0.472103  0.738719   0.972599    0.793995      0.474209   0.40042    0.777866
 0.742869  0.827304   0.0854707   0.668836   …  0.996696   0.0198933  0.128885
 0.935615  0.0279434  0.503519    0.607907      0.0668881  0.305944   0.270246
 ⋮                                          ⋱                          ⋮
 0.283703  0.300498   0.628952    0.979454   …  0.363482   0.678737   0.806071
 0.286709  0.371715   0.00598067  0.426257      0.472312   0.206808   0.625023
 0.653571  0.782553   0.912633    0.354508      0.808093   0.351514   0.160828
 0.411262  0.991678   0.832078    0.921298      0.147146   0.0679026  0.873036
 0.58339   0.274375   0.778289    0.553236      0.50523    0.895184   0.309709
 0.751978  0.226418   0.251993    0.0856854  …  0.171604   0.798678   0.247471
```

```julia
mat = rand(Cfloat, 2^8, 2^8)
```

```julia
c_sum(x::Matrix{Cfloat}) = ccall(("sum", sum_matrix_lib), Cfloat, (Ptr{Cfloat}, Cint, Cint), x, size(x, 1), size(x, 2));
```

```c
#include <stdio.h>

float sum(float *mat, int n, int m) {
  float total = 0;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      total += mat[i + j * n];
    }
  }
  return total;
}
```

▶ **What is the performance issue of this code ?**

Arithmetic intensity

Consider a program requiring `m` load / store operations with memory for `o` arithmetic operations.

- The *arithmetic intensity* is the ratio $a = o/m$.
- The arithmetic time is $t_{\text{arith}} = o/\text{frequency}$
- The data transfer time is $t_{\text{mem}} = m/\text{bandwidth} = o/(a \cdot \text{bandwidth})$

As arithmetic operations and data transfer are done in parallel, the time per iteration is

$$\max(t_{\text{arith}}, t_{\text{mem}})/o = 1/\min(\text{frequency}, a \cdot \text{bandwidth})$$

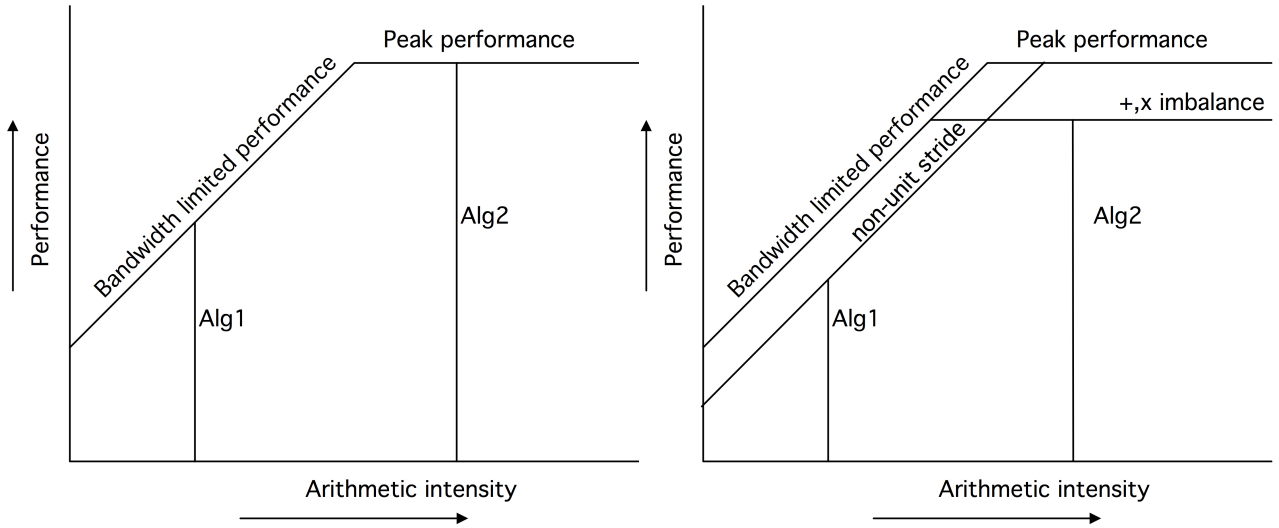
So the number of operations per second is $\min(\text{frequency}, a \cdot \text{bandwidth})$.

This piecewise linear function in a gives the *roofline model*.

Tip

See examples in [Eij10; Section 1.6.1].

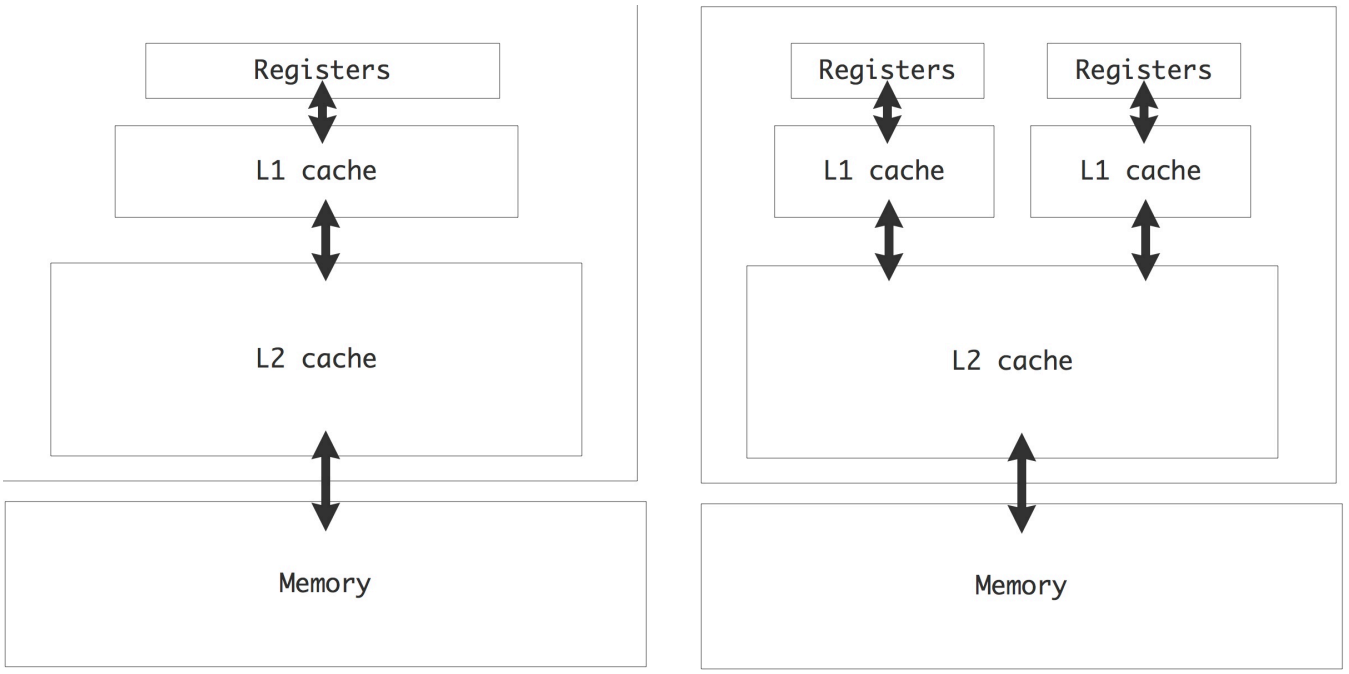
The roofline model



- *compute-bound* : For large arithmetic intensity (Alg2 in above picture), performance determined by processor characteristics
- *bandwidth-bound* : For low arithmetic intensity (Alg1 in above picture), performance determined by memory characteristics
- Bandwidth line may be lowered by inefficient memory access (e.g., no locality)
- Peak performance line may be lowered by inefficient use of CPU (e.g., not using SIMD)

[Eij10; Figure 1.16]

Cache hierarchy for a multi-core CPU



Cache coherence : Update L1 cache when the corresponding memory is modified by another core.

[Eij10; Figure 1.13]

Parallel sum

```
#include <vector>
#include <stdint.h>
#include <omp.h>
#include <stdio.h>

extern "C" {
float sum(float *vec, int length, int num_threads, int verbose) {
  float total = 0;
  omp_set_dynamic(0); // Force the value `num_threads`
  omp_set_num_threads(num_threads);
  #pragma omp parallel
  {
    int thread_num = omp_get_thread_num();
    int stride = length / num_threads;
    int last = stride * (thread_num + 1);
    if (thread_num + 1 == num_threads)
      last = length;
    if (verbose >= 1)
      fprintf(stderr, "thread id : %d / %d %d:%d\n", thread_num, omp_get_num_thre
ads(), stride * thread_num, last - 1);
    #pragma omp simd
    for (int i = stride * thread_num; i < last; i++)
      total += vec[i];
  }
  return total;
}
}
```

32869.227f0

```
@btime c_sum($vec, num_threads = 1, verbose = 0)
```

```
2.751 μs (0 allocations: 0 bytes)
```

32869.223f0

```
@btime c_sum($vec; num_threads, verbose = 0)
```

```
2.162 μs (3 allocations: 80 bytes)
```

```
32869.223f0
```

```julia
@time c_sum(vec; num_threads, verbose = 1)
```

```
thread id : 0 / 2 0:32767
thread id : 1 / 2 32768:65535
  0.000039 seconds
```

```
vec =
[0.0688847, 0.400062, 0.674264, 0.436331, 0.391714, 0.617914, 0.498138, 0.404639, 0.20417
```

```julia
vec = rand(Cfloat, 2^log_size)
```

```julia
c_sum(x::Vector{Cfloat}; num_threads = 1, verbose = 0) = ccall(("sum", sum_lib),
Cfloat, (Ptr{Cfloat}, Cint, Cint, Cint), x, length(x), num_threads, verbose);
```

Low level implementation using POSIX Threads (pthreads) covered in "LEPL1503 : Projet 3". We use the high level OpenMP library in this course.

log_size = 16

num_threads = 2

Can you spot the issue in the code ?

Many processors

```
#include <omp.h>
#include <stdio.h>

extern "C" {
void sum_to(float *vec, int length, float *local_results, int num_threads, int ve
rbose) {
  omp_set_dynamic(0); // Force the value `num_threads`
  omp_set_num_threads(num_threads);
  #pragma omp parallel
  {
    int thread_num = omp_get_thread_num();
    int stride = length / num_threads;
    int last = stride * (thread_num + 1);
    if (thread_num + 1 == num_threads)
      last = length;
    if (verbose >= 1)
      fprintf(stderr, "thread id : %d / %d %d:%d\n", thread_num, omp_get_num_thre
ads(), stride * thread_num, last - 1);
    float no_false_sharing = 0;
    #pragma omp simd
    for (int i = stride * thread_num; i < last; i++)
      no_false_sharing += vec[i];
    local_results[thread_num] = no_false_sharing;
  }
}

float sum(float *vec, int length, int num_threads, int factor, int verbose) {
  float* buffers[2] = {new float[num_threads], new float[num_threads / factor]};
  sum_to(vec, length, buffers[0], num_threads, verbose);
  int prev = num_threads, cur;
  int buffer_idx = 0;
  for (cur = num_threads / factor; cur > 0; cur /= factor) {
    sum_to(buffers[buffer_idx % 2], prev, buffers[(buffer_idx + 1) % 2], cur, ver
bose);
    prev = cur;
    buffer_idx += 1;
  }
  if (prev == 1)
    return buffers[buffer_idx % 2][0];
  sum_to(buffers[buffer_idx % 2], prev, buffers[(buffer_idx + 1) % 2], 1, verbos
e);
  return buffers[(buffer_idx + 1) % 2][0];
}
}
```

Benchmark

If we have many processors, we may want to speed up the last part as well:

32869.223f0

```julia
@time many_sum(vec; base_num_threads, factor, verbose = 1)
```

```
thread id : 0 / 2 0:32767
thread id : 1 / 2 32768:65535
thread id : 0 / 1 0:1
  0.000082 seconds
```

32731.078f0

```julia
@btime c_sum($many_vec)
```

```
  2.734 μs (0 allocations: 0 bytes)
```

32731.078f0

```julia
@btime many_sum($many_vec; base_num_threads, factor)
```

```
  6.943 μs (3 allocations: 80 bytes)
```

many_vec =

▸ [0.129244, 0.706291, 0.132355, 0.0879626, 0.708648, 0.621902, 0.418827, 0.375107, 0.58935

```julia
many_vec = rand(Cfloat, 2^many_log_size)
```

```julia
many_sum(x::Vector{Cfloat}; base_num_threads = 1, factor = 2, verbose = 0) =
ccall(("sum", many_sum_lib), Cfloat, (Ptr{Cfloat}, Cint, Cint, Cint, Cint), x,
length(x), base_num_threads, factor, verbose);
```

many_log_size = 16

base_num_threads = 2

factor = 2

Amdahl's law

Speed-up and efficency

Def: Speed-up

$$S_p = \frac{T_1}{T_p}$$

Def: Efficiency

$$E_p = \frac{S_p}{p}$$

Let T_p bet the time with p processes

- $E_p > 1$ → Unlikely
- $E_p = 1$ → Ideal
- $E_p < 1$ → Realistic

Amdahl's law

- F_s : Fraction of T_1 that is sequential
- $F_p = 1 - F_s$: Fraction of T_1 that is parallelizable

$$T_p = T_1 F_s + T_1 F_p / p$$

$$S_p = \frac{1}{F_s + F_p/p} \qquad E_p = \frac{1}{pF_s + F_p}$$

$$\lim_{p \to \infty} S_p = \frac{1}{F_s}$$

Application to parallel sum

The first `sum_to` takes n/p operations. Assuming `factor` is `2`, there is one operation for each of the $\log_2(p)$ subsequent `sum_to`.

$$
\begin{aligned}
T_1 &= n \\
T_p &= n/p + \log_2(p) \\
S_p &= \frac{1}{1/p + \log_2(p)/n} \qquad E_p = \frac{1}{1 + p\log_2(p)/n}
\end{aligned}
$$

▶ **How to get $1/F_s = \lim_{p\to\infty} S_p$?**

```
using PlutoUI, PlutoUI.ExperimentalLayout, HypertextLiteral, Luxor, StaticArrays,
BenchmarkTools, PlutoTeachingTools, Markdown, SimpleClang
```

```
0.2
```

```
BenchmarkTools.DEFAULT_PARAMETERS.seconds = 0.2
```

```
img (generic function with 3 methods)

definition (generic function with 1 method)
```